

Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** Online Bookstore
- **Name:** Yakshagaraj A G

2. Project Overview

The Online Bookstore aims to provide users with a seamless and intuitive platform to browse, search, filter, review, and purchase books online.

It also provides an Admin Dashboard for managing books, categories, and inventory.

Goals:

- Deliver a high-performance, user-friendly digital bookstore
- Enable fast search and categorization
- Provide cart, wishlist, and order management
- Support admin-side CRUD operations
- Implement secure user authentication

Key Features:

- User authentication (JWT-based)
- Book listing & details page
- Search and advanced filtering
- Add to Cart & Wishlist
- Order placement & history
- User reviews & ratings
- Admin: Add/Edit/Delete books
- Inventory & category management

3. Architecture

Frontend (react)

- Built using React.js
- Component-based modular structure
- Responsive UI using TailwindCSS
- API communication via Axios

Backend (Node.js + Express.js)

- RESTful API architecture
- Endpoints for authentication, books, cart, wishlist, reviews, orders
- Middleware for authorization, validation & error handling
- Secure password encryption
- JWT-based access control

4. Setup Instructions

Prerequisites:

- Node.js (LTS version)
- MongoDB or MongoDB Atlas
- VS Code

Installation:

1. Install dependencies

Cd client

Npm install

Cd ../server

Npm install

2. Configure environment variables (.env)

MONGO_URI

JWT_SECRET

CLIENT_URL

5. Folder Structure

- **Client:** Describe the structure of the React frontend.
- **Server:** Explain the organization of the Node.js backend.

```
Bookstore/
├── client/      # React app
│   ├── src/components/ (Navbar, BookCard, CartModal)
│   ├── src/pages/    (Home, BookDetails, Admin)
│   └── src/services/api.js
└── server/     # Node/Express
    ├── models/   (Book.js, User.js, Order.js)
    └── routes/   (books.js, auth.js)
```

└─ middleware/auth.js [file:1]

6. Running the Application

commands to start the frontend and backend servers locally.

- **Frontend:** npm run dev in the client directory.
- **Backend:** nodemon index.js in the server directory.

7. API Documentation

AUTH ROUTES:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me

BOOK ROUTES:

- GET /api/books
- GET /api/books/:id
- POST /api/books
- PUT /api/books/:id
- DELETE /api/books/:id

ORDER ROUTES:

- POST /api/orders
- GET /api/orders/my-orders
- GET /api/orders

8. Authentication

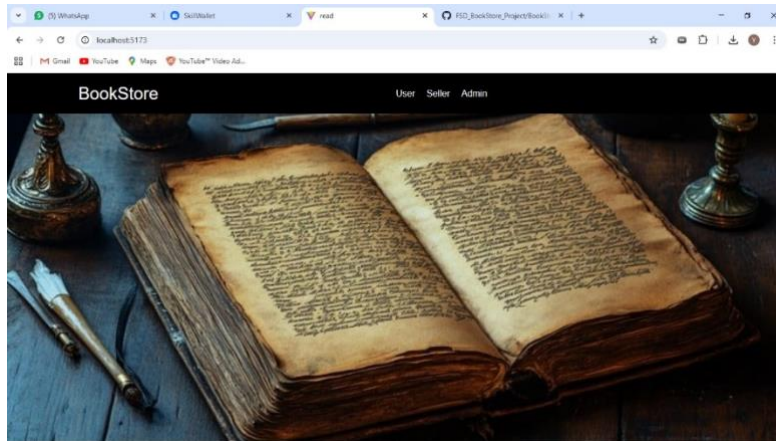
JWT-based authentication with protected routes and middleware for secure access control.

- JWT authentication for secure access
- Tokens stored in HTTP-only cookies or local storage
- Role-based access (User / Admin).

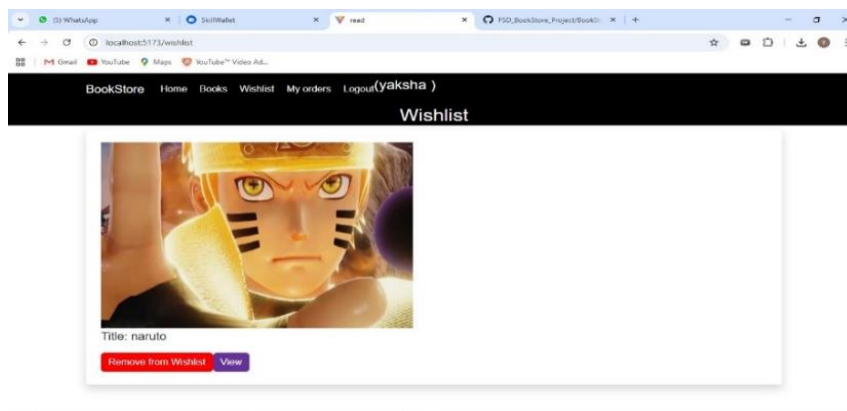
9. User Interface

Screenshot:

Home page:



Wishlist:



Seller list:

slno	Userid	User name	Email	Operation
1	69261a0e0e836a58b9128d7	noah	noah@gmail.com	View Products
2	69267aa0bc5187d953c7d969	yaksha	yaksha26@outlook.com	View Products
3	6926d249bc5187d953c7d904	sherine	ivy@gmail.com	View Products
4	6933b33da3d4ba0d4091247	noah	seller@gmail.com	View Products
5	6933f4f8a3d4ba0d4091212	noah	yaksha@gmail.com	View Products

10. Testing

Manual testing via Postman for APIs. Frontend tested in Chrome/Firefox. Edge cases: empty cart, invalid login, duplicate reviews. Unit tests planned for controllers

11. Screenshots or Demo :

demo link:

<https://drive.google.com/drive/folders/19mpKBphmZyt3fZAt7zMzah6dxc>

[KSRobM](#)

12. Known Issues

- Image upload size limits not enforced
- Search lacks fuzzy matching
- No email verification
- No forgot password option

13. Future Enhancements

- Payment gateway (Stripe/Razorpay)
- Book recommendations (collaborative filtering)
- Discount coupons
- Audiobook integration
- AI chatbot for suggestions